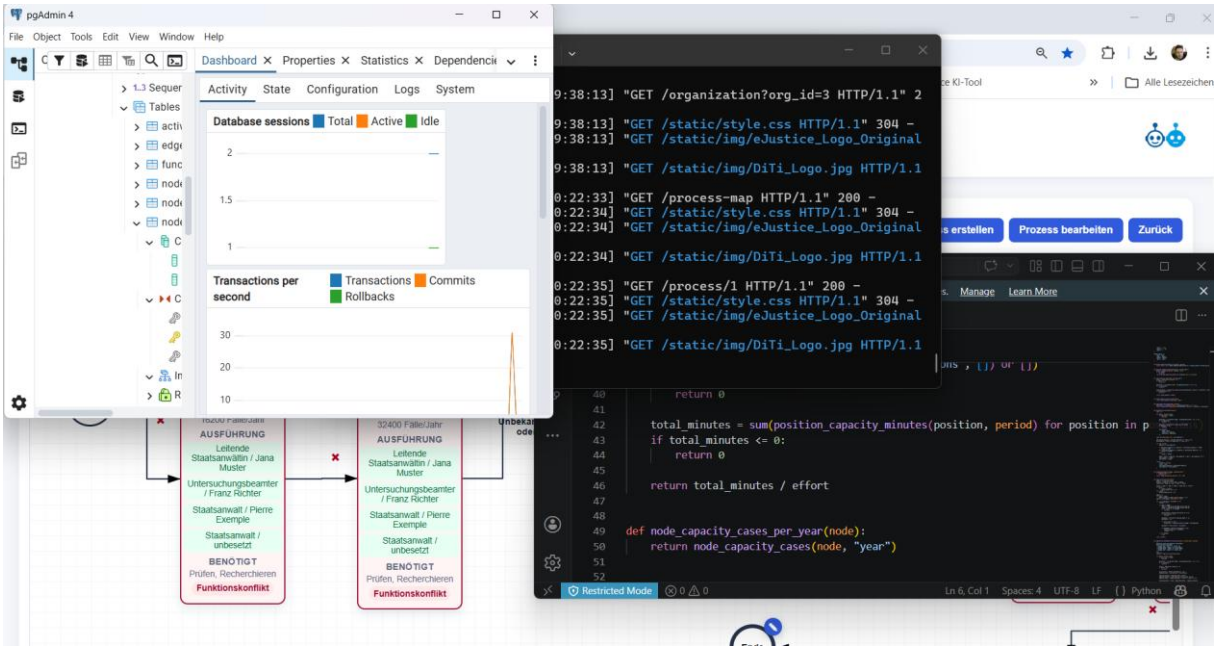
Technische Dokumentation Prozess-Simulator



Inhalt

1 Systemübersicht 2
2 Datenmodell (vollständig) 2
3 Routing / Controller Layer 4
4 Berechnungslogik (formal) 5
5 Frontend Architektur 6
6 Validierungslogik 6
7 Persistenzverhalten 7
8 Performance 7
9 Bekannte technische Limitationen 7
10 Erweiterungspunkte (technisch) 8
11 Fazit (technisch) 8

M. Balmer * K. Brönnimann * N. Gloor * D. Kettiger * X. Schütz * T. Hügli

Open Legal Lab 25. – 27. April 2026 Magglingen

1 Systemübersicht

1.1 Ziel des Systems

Der Prozess-Simulator ist ein **zustandsbehaftetes, datenbankgestütztes Websystem**, das:

- BPMN-ähnliche Prozessgraphen persistiert
- Organisationsstrukturen modelliert
- Ressourcen (Personen, FTE) zuweist
- deterministische Simulationen auf Graphen ausführt

1.2 Architektur

Client (Browser)
↓
Flask Web Layer (app.py)
↓
Service / Calculation Layer (dashboard.py)
↓
ORM Layer (SQLAlchemy)
↓
PostgreSQL

1.3 Architekturstil

- Monolithisch
- Server-side rendering (Jinja2)
- Stateful DB
- Kein API-first Design (klassisches MVC)

2 Datenmodell (vollständig)

2.1 Process

```
Process (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  parent_process_id INTEGER NULL,
  x FLOAT,
  y FLOAT
)
```

Constraints

- parent_process_id → Process.id (self-reference)
- Kein Zyklus-Check implementiert (technische Limitation)

2.2 Node

```
Node (
  id SERIAL PRIMARY KEY,
  process_id INTEGER NOT NULL,
  type TEXT NOT NULL,
  name TEXT,
  effort_minutes FLOAT,
  x FLOAT,
```

```
    y FLOAT,
    subprocess_id INTEGER NULL
)
```

- **Typen**
- start
- task
- xor
- subprocess
- **Constraints**
- process_id → Process.id
- subprocess_id → Process.id

2.3 Edge

```
Edge (
    id SERIAL PRIMARY KEY,
    source_node_id INTEGER NOT NULL,
    target_node_id INTEGER NOT NULL,
    probability_percent FLOAT NULL
)
```

- **Regeln**
- Nur bei source_node.type == 'xor' relevant
- Summe aller Outgoing = 100% (nicht DB-validiert, nur logisch)

2.4 Organisation

- **Organization**

```
Organization (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL
)
```

- **OrgUnit**

```
OrgUnit (
    id SERIAL PRIMARY KEY,
    organization_id INTEGER NOT NULL,
    parent_id INTEGER NULL,
    name TEXT NOT NULL
)
```

- **Position (Stelle)**

```
Position (
    id SERIAL PRIMARY KEY,
    org_unit_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    person_id INTEGER NULL
)
```

- **Person**

```
Person (
   id SERIAL PRIMARY KEY,
   name TEXT NOT NULL,
   fte FLOAT NOT NULL,
   salary FLOAT
)
```

2.5 Rollen & Funktionen

```
Role (
   id SERIAL PRIMARY KEY,
   name TEXT
)

Function (
   id SERIAL PRIMARY KEY,
   name TEXT,
   role_id INTEGER
)
```

2.6 Mappings

- **Person ↔ Role**

```
person_roles (
   person_id,
   role_id
)
```

- **Person ↔ Function**

```
person_functions (
   person_id,
   function_id
)
```

- **Node ↔ Function (Anforderungen)**

```
node_functions (
   node_id,
   function_id
)
```

- **Node ↔ Position (Ressourcenzuordnung)**

```
node_positions (
   node_id,
   position_id
)
```

3 Routing / Controller Layer

3.1 Prozess

GET /processes
GET /processes/<id>
POST /processes/create
POST /processes/<id>/delete

3.2 Nodes

POST /process/<process_id>/nodes/add
POST /process/<process_id>/nodes/<node_id>/edit
POST /process/<process_id>/nodes/<node_id>/delete

3.3 Edges

POST /edges/create
POST /edges/<id>/delete

3.4 Dashboard

GET /dashboard

Query Params:

- tab
- op_cases
- op_period

4 Berechnungslogik (formal)

4.1 Grundkonstanten

FTE_YEAR_HOURS = 2100
FTE_YEAR_MINUTES = 126000

4.2 Zeitraum-Funktion

```
capacity_minutes(fte, period) =
   fte * FTE_YEAR_MINUTES * factor(period)

factor(period) =
   day   → 1/220
   week  → 1/52
   month → 1/12
   year  → 1
```

4.3 Kapazität pro Node

```
capacity_cases(node) =
   sum(position_capacity_minutes) / effort_minutes
```

4.4 Besuchsfaktor (Graph Traversal)

- **Initialisierung**

f(start) = 1.0

- **Propagation**

```
f(target) = max(
   f(source) * multiplier(edge)
)
```

- **XOR**

```
multiplier(edge) =
   edge.probability_percent / 100
```

4.5 Operative Berechnung

```
expected_cases(node) =
   input_cases * visit_factor(node)

required_minutes =
   expected_cases * effort_minutes

missing_minutes =
   max(0, required_minutes - capacity_minutes)
```

4.6 Entscheidungslogik

```
if missing_minutes > 0:
   node.status = "overloaded"
else:
   node.status = "ok"
```

5 Frontend Architektur

5.1 BPMN Editor

Technologie:

- HTML Canvas / DOM
- JS Event Handling
- **Funktionen**
- Drag Nodes
- Create Edge (2-click)
- Delete Edge
- Edit Node
- Open Subprocess

5.2 Dashboard

- **Tabs**
- strategic
- operational
- **Rendering**
- Jinja2 Server Rendering
- CSS Grid (3 Spalten)

5.3 State Handling

- GET Parameter gesteuert
- Kein Client-State-Management

6 Validierungslogik

6.1 XOR

Soll:

sum(probabilities) = 100%

Ist:

- keine harte Validierung
- UI-abhängig

6.2 Funktionen

allowed(person, node) =
  node.functions ⊆ (person.functions ∪ role.functions)

7 Persistenzverhalten

7.1 Koordinaten

- Nodes und Prozesse speichern x, y
- Zustand bleibt stabil nach Reload

7.2 Cascading Deletes

Nicht vollständig konsistent:

Problem:

Process → Node → Edge

muss manuell gelöscht werden

8 Performance

8.1 Komplexität

- **Traversal**

O(N * E)

- **Dashboard**

O(P * N)

8.2 Bottlenecks

- viele Nodes → langsame Iteration
- kein Caching
- kein Lazy Loading

9 Bekannte technische Limitationen

9.1 Graph

- keine Zyklenkontrolle
- keine Parallel-Gateways (AND)

9.2 DB

- keine Constraints für XOR-Summe
- keine Historisierung

9.3 UI

- kein Undo
- kein Versioning
- kein Multi-User Locking

10 Erweiterungspunkte (technisch)

10.1 Kurzfristig

- XOR Validierung auf DB-Ebene
- Foreign Key Cascades sauber definieren
- API Layer einführen

10.2 Mittelfristig

- Graph Engine (networkx)
- Simulation Engine (Discrete Event)

10.3 Langfristig

- Microservice Split
- Event Streaming (Kafka)
- Optimierungsalgorithmen

11 Fazit (technisch)

Das System ist:

- deterministisch
- graph-basiert
- relational modelliert
- UI-getrieben